#Skincare Ecommerce: Performance Review & Outlook *Data Analysis & Forecasting Report*

1. Introduction

The company operates a global online store selling skincare, beauty, and related products, and management seeks clearer visibility into where value is created and how to optimise growth. This project is going to analyse three core areas of its historical order data (1 Jan 2020 – 31 Dec 2023):

- Market & Customer Value Drivers: Identifying the regions, markets, and customer segments that contribute the most sales and profit.
- Product & Discount Profitability: Assessing how product categories and discount levels influence revenue and margin.
- Sales Forecasting: Building a time-series model to project monthly sales for the next 6 months.

This report aims to provide clear, data-driven insights to guide strategic decisions in marketing investment, assortment planning, regional expansion, and inventory management. The dataset used for this analysis comes from the company's global ecommerce transactional records.

2. Data exploration

#####2.1 Import libraries & Load the dataset

```
#Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

#Load the dataset
data = pd.read_csv("/Volumes/python/trang_project/trang_do_python_project/Global skincare and Beauty e-store_E-commerce Analysis_English.csv")
data.head()
```

```
   Row ID                Order ID Order Date  ... Sales Discount Profit
0   46682   IZ-2012-LW699061-40911  3/01/2020  ...   600      0.0  300.0
1   10124  US-2012-BT1130518-40912  4/01/2020  ...    40      0.6   -4.0
2    9067  MX-2012-AW1093031-40912  4/01/2020  ...   151      0.0   75.5
3     130  MX-2012-BT1130531-40912  4/01/2020  ...    40      0.0   20.0
4   24072  IN-2012-KM1666027-40914  6/01/2020  ...    11      0.0    5.5

[5 rows x 19 columns]
```

#####2.2 Basic info about the data

```
#Number of rows and columns
data.shape
```

```
(51290, 19)
```

```
#Column names and data types
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51290 entries, 0 to 51289
Data columns (total 19 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Row ID             51290 non-null  int64
 1   Order ID           51290 non-null  object
 2   Order Date         51290 non-null  object
 3   Customer ID        51290 non-null  object
 4   Segment            51290 non-null  object
```

```
 5   City               51290 non-null  object
 6   State              51290 non-null  object
 7   Country            51290 non-null  object
 8   Country latitude   51290 non-null  float64
 9   Country longitude  51290 non-null  float64
 10  Region             51290 non-null  object
 11  Market             51290 non-null  object
 12  Subcategory        51290 non-null  object
 13  Category           51290 non-null  object
 14  Product            51290 non-null  object
 15  Quantity           51290 non-null  int64
 16  Sales              51290 non-null  int64
 17  Discount           51290 non-null  float64
 18  Profit             51290 non-null  float64
dtypes: float64(4), int64(3), object(12)
memory usage: 7.4+ MB
```

All columns have 0 missing values, which simplifies preparation.

Dataset includes:

- 51,290 rows, 19 columns.
- Each row is a product line in an order, with information on who bought, where, what, how much, discount, and profit.

Columns include:

- Order & customer: Row ID, Order ID, Order Date, Customer ID, Segment
- Geography: City, State, Country, Country latitude, Country longitude, Region, Market
- Product: Category, Subcategory, Product
- Transaction: Quantity, Sales, Discount, Profit

```
#Summary stats for numeric columns
data.describe()
```

	Row ID	Country latitude	...	Discount	Profit
count	51290.00000	51290.000000	...	51290.000000	51290.000000
mean	25645.50000	24.506298	...	0.142908	20.772346
std	14806.29199	24.119393	...	0.212280	83.582680
min	1.00000	-40.900557	...	0.000000	-1746.000000
25%	12823.25000	12.879721	...	0.000000	0.000000
50%	25645.50000	35.861660	...	0.000000	7.200000
75%	38467.75000	38.963745	...	0.200000	25.000000
max	51290.00000	61.924110	...	0.850000	1820.000000

[8 rows x 7 columns]

The dataset contains 51,290 order lines, mostly small to medium in size, with typical quantities between 2 and 7 units per order. Sales are strongly right-skewed: while the median order value is only 56, a small number of high-value orders (up to 3,940) push the average up to 127. Discounts are applied unevenly, at least half of the orders receive no discount at all, but some transactions get very deep discounts of up to 85%, indicating a highly variable discounting strategy.

Profit is also very volatile. Although the average profit per order is about 20.8, the median is much lower at 7.2, and there are both heavily loss-making orders (down to -1,746) and highly profitable ones (up to 1,820). This combination of skewed sales, inconsistent discounting, and wide profit variation suggests that a small subset of orders and discount decisions may significantly influence overall profitability and should be investigated further (e.g. by product, region, or customer segment).

```
#Check for potential duplicates
duplicate_rows = data.duplicated(subset=["Row ID"]).sum()
duplicate_order_lines = data.duplicated(subset=["Order ID", "Product"]).sum()
duplicate_rows, duplicate_order_lines
```

```
(np.int64(0), np.int64(102))
```

```
duplicate_order_lines = data[data.duplicated(subset=["Order ID", "Product"], keep=False)]

duplicate_order_lines = duplicate_order_lines.sort_values(["Order ID", "Product"])

duplicate_order_lines
```

```
       Row ID                  Order ID  Order Date  ... Sales Discount  Profit
24900   35223  CA-2012-BN11515140-41168  16/09/2020  ...    36      0.0    5.40
28250   35222  CA-2012-BN11515140-41168  16/09/2020  ...    54      0.0    8.10
32811   40605  CA-2012-CA12265140-40992  24/03/2020  ...   160      0.0   16.00
42404   40606  CA-2012-CA12265140-40992  24/03/2020  ...   240      0.0  120.00
42427   34573  CA-2012-GM14695140-41003   4/04/2020  ...    51      0.0   25.50
...       ...                       ...         ...  ...   ...      ...     ...
32336   35886  US-2015-NL18310140-42311   3/11/2023  ...    75      0.0   11.25
26861   39768  US-2015-PN18775140-42118  24/04/2023  ...    16      0.2   -0.80
35006   39767  US-2015-PN18775140-42118  24/04/2023  ...     8      0.2   -0.80
15911   32399  US-2015-RP19390140-42116  22/04/2023  ...    21      0.8   -8.40
15912   32400  US-2015-RP19390140-42116  22/04/2023  ...    21      0.8   -8.40

[204 rows x 19 columns]
```

No duplicate Row IDs were found; however, 102 duplicated Order ID + Product combinations were identified. These should be removed to prevent overstating sales and quantities in the analysis.

```
#Drop duplicates
data = data.drop_duplicates(subset=["Order ID", "Product"], keep="first")

duplicate_order_lines = data.duplicated(subset=["Order ID", "Product"]).sum()
duplicate_order_lines
```

```
np.int64(0)
```

```
n_products = data["Product"].nunique()
top_products = data["Product"].value_counts().head(5)
n_products, top_products
```

```
(3577,
 Product
 Herbal Essences Bio                              335
 Neutrogena Hydro Boost Gel Cream                 227
 Essie Nail Polish Aruba Blue Shimmering Cobalt    92
 Head & Shoulders Classic Clean Conditioner        90
 Redken Color Extend Magnetics Conditioner         84
 Name: count, dtype: int64)
```

The catalogue spans 3577 products, with the top 5 SKUs are Herbal Essences Bio, Neutrogena Hydro Boost Gel Cream, Essie Nail Polish Aruba Blue Shimmering Cobalt, Head & Shoulders Classic Clean Conditioner & Redken Color Extend Magnetics Conditioner

```
top_products_sales = (
    data.groupby(["Product", "Category"])["Sales"]
        .sum()
        .reset_index()
        .sort_values("Sales", ascending=False)
        .head(5)
)

top_products_sales
```

```
                     Product     Category  Sales
5045       Herbal Essences Bio    Body care  29108
```

```
10990    Rose Gold Petal Studs             Body care  21038
11591      Silver Embrace Ring             Body care  15390
5048       Herbal Essences Bio  Home and Accessories  14972
4817   Golden Feather Earrings             Body care  14325
```

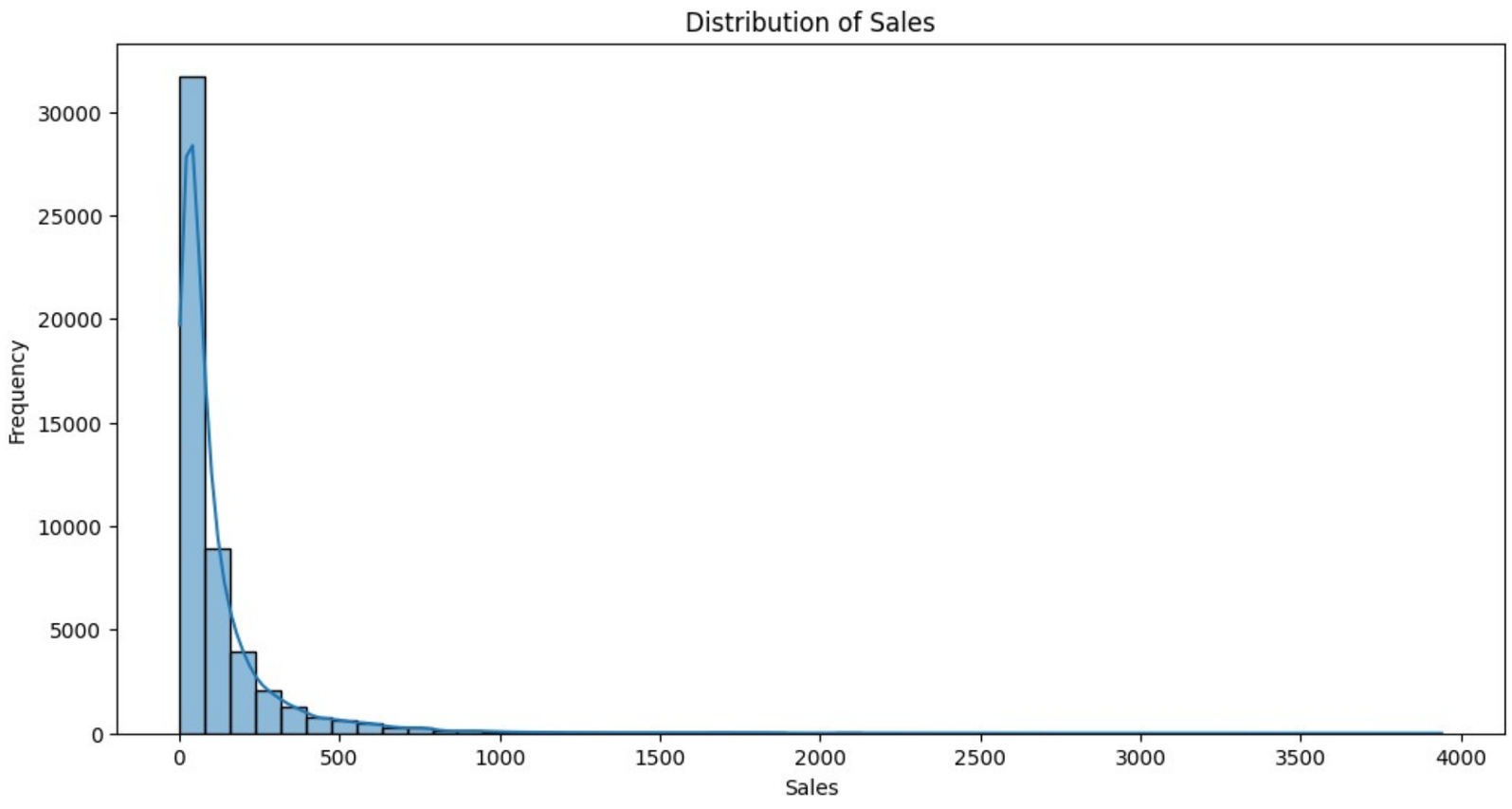
The top-selling products in the dataset include Herbal Essences Bio, Rose Gold Petal Studs, Silver Embrace Ring, and Golden Feather Earrings. These items currently appear under categories such as Body care and Home and Accessories. However, several products are incorrectly categorized — for example, jewellery items appearing under Body care and Herbal Essences Bio listed under two different categories.

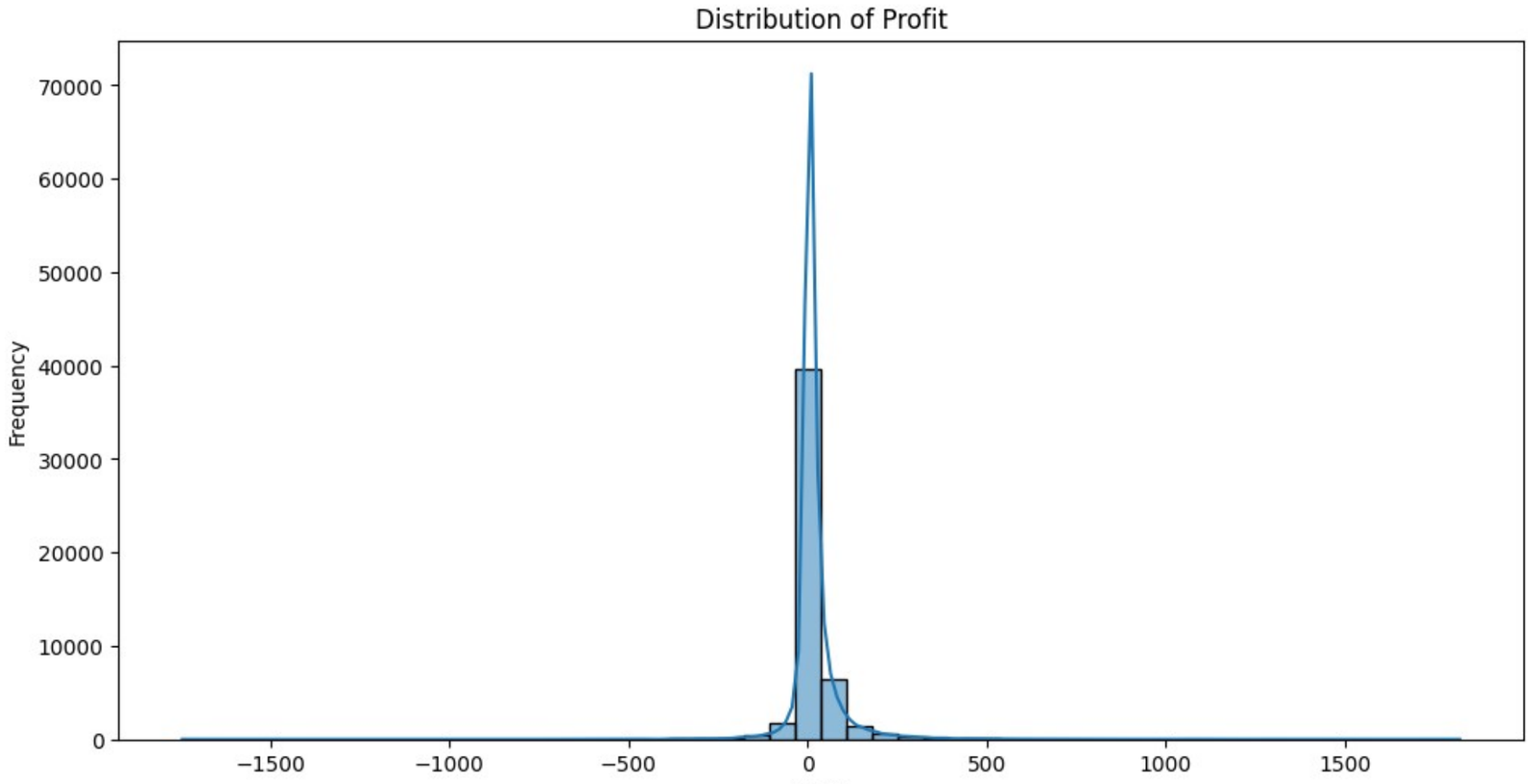
These category misplacements will be corrected later in the data preparation stage to ensure accurate analysis.

#####2.3 Check distribution shape and outliers of the dataset using histograms and boxplots

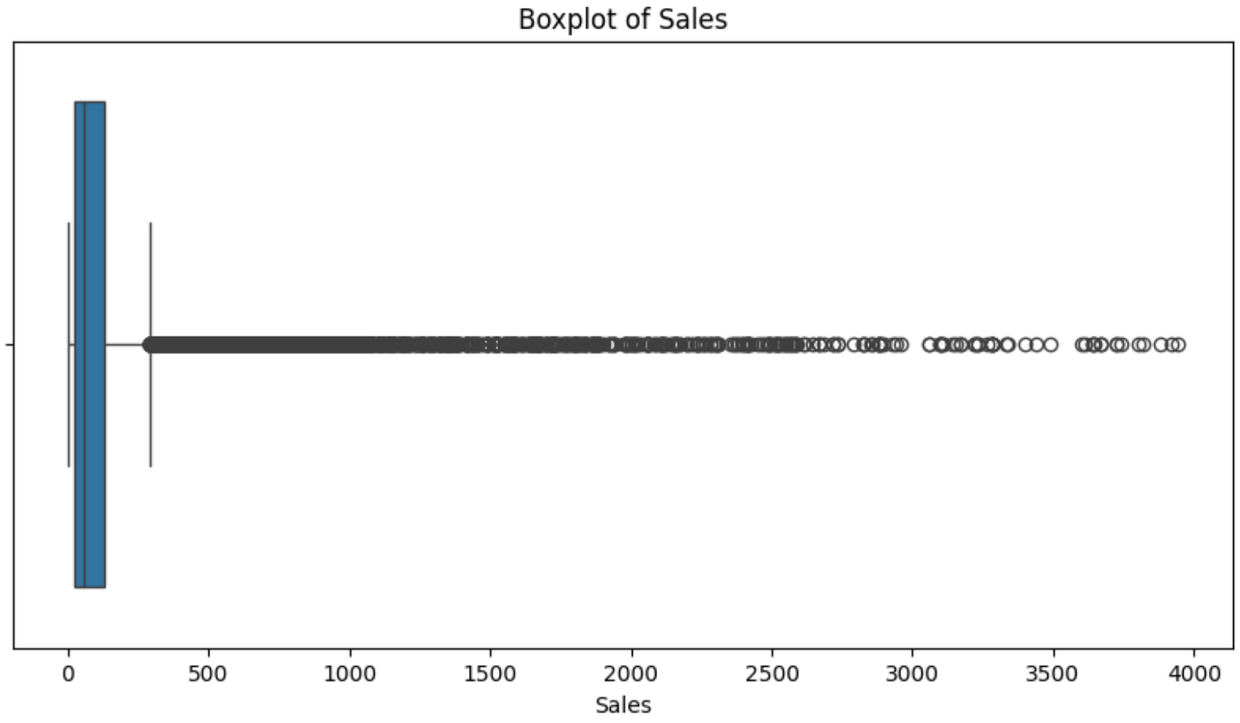
```
#Histogram for numeric variables
numeric_cols = ["Sales", "Profit"]

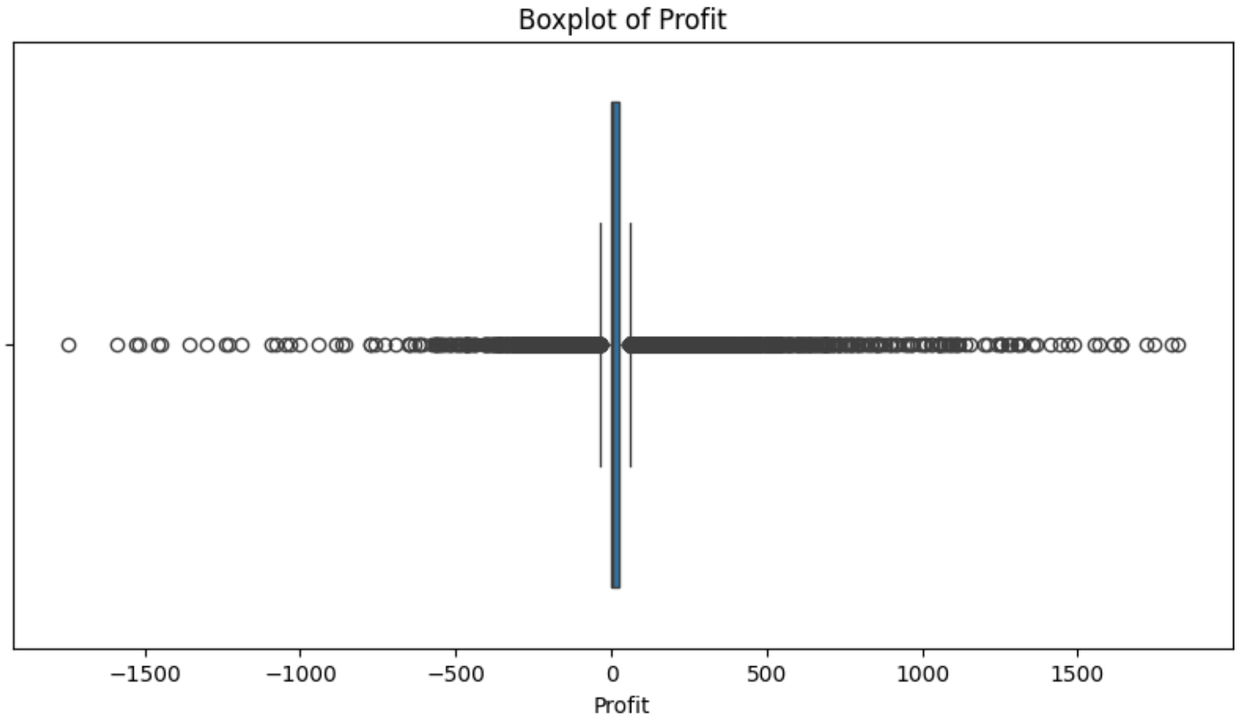
for col in numeric_cols:
    plt.figure(figsize=(12,6))
    sns.histplot(data[col], bins=50, kde=True)
    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.show()
```






```
#Boxplot to spot outliers
for col in numeric_cols:
    plt.figure(figsize=(10,5))
    sns.boxplot(x=data[col])
    plt.title(f"Boxplot of {col}")
    plt.xlabel(col)
    plt.show()
```





Sales is highly right-skewed, with most orders falling between 0–300, indicating that small-value transactions dominate. A long tail up to ~4000 shows the presence of a few very large orders, which create significant variability and many high-value outliers in the boxplot. These large orders may reflect bulk/corporate purchases, promotions, or data entry issues.

Profit is concentrated around 0–50, meaning most orders generate only small margins. However, there are extreme outliers on both ends, including very high-profit orders (+1500+) and large losses (–1700). The boxplot is stretched due to these extremes. High-profit orders may come from premium items or high-value bundles, while large negative-profit orders indicate potential issues such as heavy discounts, returns, shipping costs.

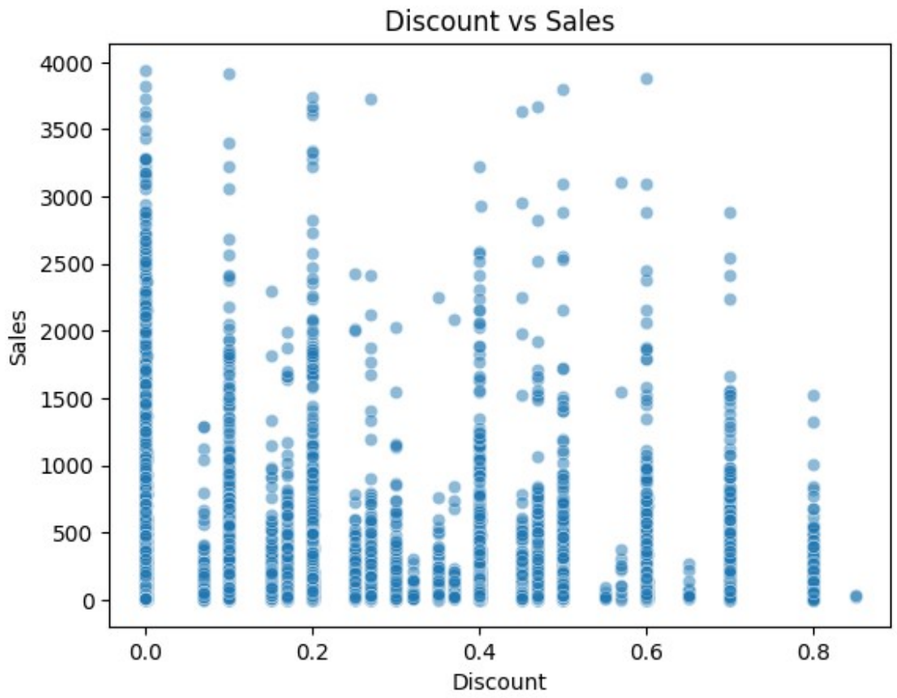
#####2.4 Checking Relationships Between Variables

```
#Relationships Between Sales vs Profit
plt.figure()
sns.scatterplot(data, x="Sales", y="Profit", alpha=0.5)
plt.title("Sales vs Profit")
plt.show()
```



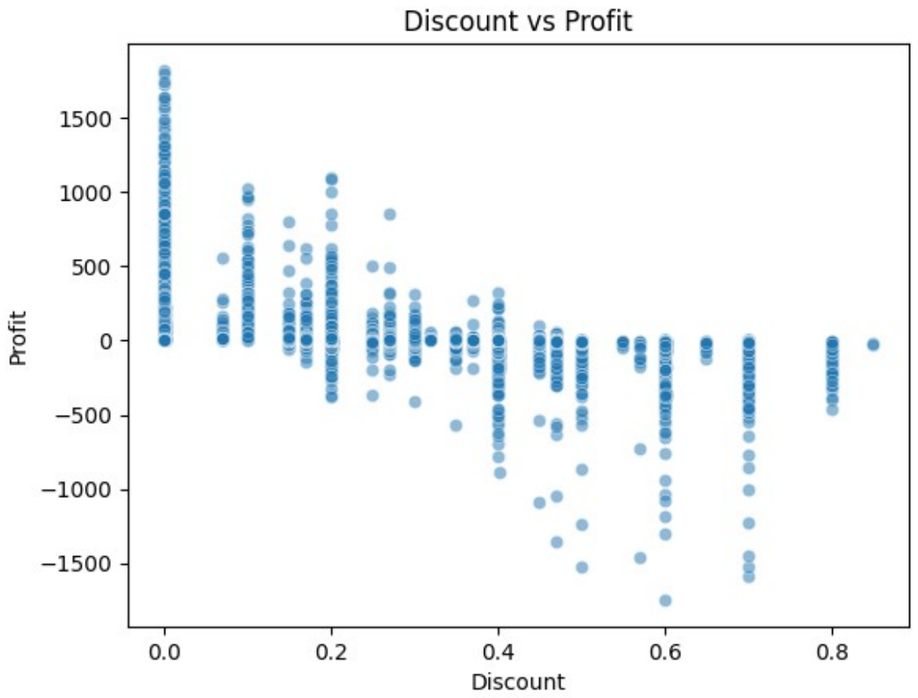
Sales and Profit show a positive relationship overall, but with significant variability. While higher sales often lead to higher profit, several high-sales orders generate large losses, indicating potential issues with discounting, cost structure, or returns. Most transactions cluster at low sales and low profit, while high-value orders carry both high opportunity and high risk.

```
#Discount impact on Sales
plt.figure()
sns.scatterplot(data, x="Discount", y="Sales", alpha=0.5)
plt.title("Discount vs Sales")
plt.show()
```



Discount levels show no meaningful relationship with sales amounts. High-value sales appear across all discount levels, including orders with no discount. Deep discounts (>60%) do not generate higher sales but significantly increase profit risk. This suggests that the current discount strategy may not effectively drive revenue and should be reviewed.

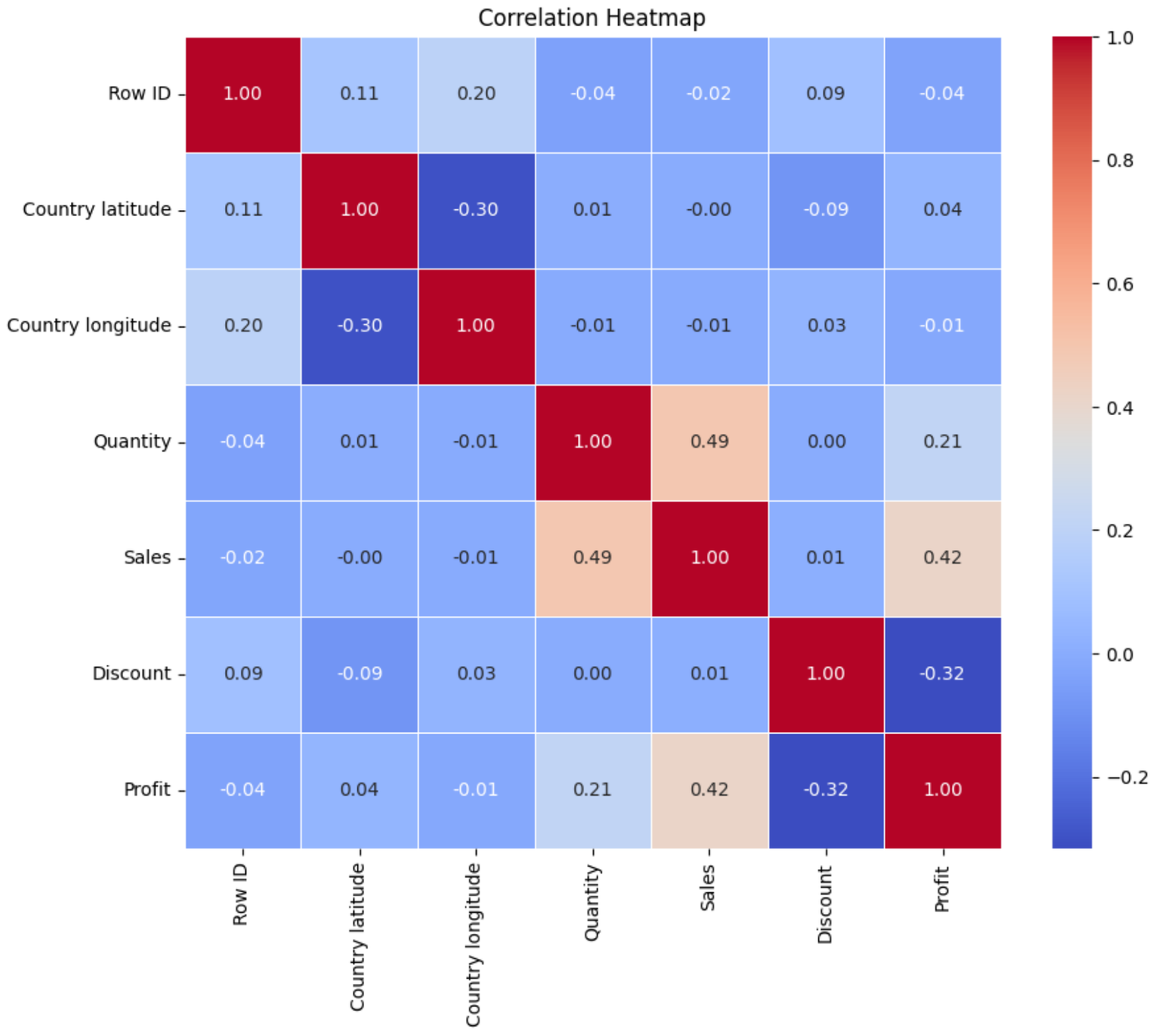
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
#Discount impact on Profit
plt.figure()
sns.scatterplot(data, x="Discount", y="Profit", alpha=0.5)
plt.title("Discount vs Profit")
plt.show()
```



There is a strong negative relationship between discount and profit. Low-discount orders generate the highest profit, while high-discount orders (40–80%) consistently produce substantial losses. This indicates that the current discount strategy is eroding margins without driving higher sales, and requires immediate review and optimization.

```
#Correlation Heatmap (Sales, Profit, Discount, Quantity)
num_data = data.select_dtypes(include=["number"])
corr = num_data.corr()

plt.figure(figsize=(10,8))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```



Correlation heatmap shows that Sales and Quantity are moderately correlated, and Sales is moderately correlated with Profit. Discount shows almost no relationship with Sales but has a clear negative correlation with Profit (–0.32), indicating that discounting reduces profitability without effectively driving revenue. This suggests that discount strategies should be reviewed or optimized.

3. Data preparation

#####3.1 Converting Date Fields & Generating Summary Metrics

```
#Converting Date Fields
data["Order Date"] = pd.to_datetime(data["Order Date"],format="mixed",dayfirst=True)
start_date = data["Order Date"].min()
end_date = data["Order Date"].max()
#Generating Summary Metrics
n_orders = data["Order ID"].nunique()
n_customers = data["Customer ID"].nunique()
```

```
total_sales = data["Sales"].sum()
total_profit = data["Profit"].sum()
aov = total_sales / n_orders
profit_margin = total_profit / total_sales * 100

summary_stats = {
    "Start date": start_date,
    "End date": end_date,
    "Rows": len(data),
    "Unique orders": n_orders,
    "Unique customers": n_customers,
    "Total sales": total_sales,
    "Total profit": total_profit,
    "Average order value": aov,
    "Overall profit margin (%)": profit_margin
}
summary_stats
```

```
{'Start date': Timestamp('2020-01-01 00:00:00'),
 'End date': Timestamp('2023-12-31 00:00:00'),
 'Rows': 51290,
 'Unique orders': 25728,
 'Unique customers': 17415,
 'Total sales': np.int64(6517674),
 'Total profit': np.float64(1065413.65),
 'Average order value': np.float64(253.32999067164178),
 'Overall profit margin (%)': np.float64(16.34653175350593)}
```

Key numbers (2020–2023):

- Total sales: $6,517,674
- Total profit: $1,065,414
- Unique orders: 25,728
- Unique customers: 17,415
- Average order value (AOV): $253.33
- Overall profit margin: 16.3%

From 2020–2023, the business generated $6.5M in sales and $1.06M in profit, with a healthy 16.3% margin. Strong AOV ($253) and a broad customer base (17k+) reflect robust demand and high-value transactions. While overall performance is solid, earlier findings show that certain products and discount issues create losses, needing strategy to further optimize profitability.

#####3.2 Data Cleaning: Removing Unnecessary Columns

Dropping the Latitude and Longitude columns as they are not needed for this analysis. We already have the Country field, which provides sufficient geographic information for sales reporting.

```
# Drop latitude/longitude columns
data = data.drop(
    ["Country latitude", "Country longitude"],
    axis=1,
    errors="ignore"
)
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51290 entries, 0 to 51289
Data columns (total 17 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Row ID       51290 non-null  int64
```

```
 1   Order ID     51290 non-null  object
 2   Order Date   51290 non-null  datetime64[ns]
 3   Customer ID  51290 non-null  object
 4   Segment      51290 non-null  object
 5   City         51290 non-null  object
 6   State        51290 non-null  object
 7   Country      51290 non-null  object
 8   Region       51290 non-null  object
 9   Market       51290 non-null  object
 10  Subcategory  51290 non-null  object
 11  Category     51290 non-null  object
 12  Product      51290 non-null  object
 13  Quantity     51290 non-null  int64
 14  Sales        51290 non-null  int64
 15  Discount     51290 non-null  float64
 16  Profit       51290 non-null  float64
dtypes: datetime64[ns](1), float64(2), int64(3), object(11)
memory usage: 6.7+ MB
```

#####3.3 Data Cleaning: Correcting Mislabelled Categories

During the data exploration stage, we identified that many jewellery and accessory items were mistakenly labelled under Body care. This misclassification:

- increases Body care sales
- makes Home & Accessories appear unprofitable

Fixing the Issue

```
#Define accessory keywords:
accessory_keywords = [
    "necklace", "earring", "bracelet", "ring", "anklet",
    "chain", "jewelry", "jewellery", "pendant", "bangle", "charm", "earrings", "necklaces"
]

#Detect & correct misclassified items
data_orig = data.copy()

mask_body = data["Category"] == "Body care"
mask_accessory_name = data["Product"].str.lower().str.contains("|".join(accessory_keywords))

mis_idx = data.index[mask_body & mask_accessory_name]

print("Misclassified rows:", len(mis_idx))

#Reassign to correct category
data.loc[mis_idx, "Category"] = "Home and Accessories"
data.loc[mis_idx, "Subcategory"] = "Accessories"
```

```
Misclassified rows: 1674
```

1674 misclassified accessories were identified and corrected before making any serious category conclusions

######Check Impact on Categories

```
cat_before = data_orig.groupby("Category")[["Sales","Profit"]].sum()
cat_before["Margin_%"] = cat_before["Profit"] / cat_before["Sales"] * 100

cat_after = data.groupby("Category")[["Sales","Profit"]].sum()
cat_after["Margin_%"] = cat_after["Profit"] / cat_after["Sales"] * 100
```

```
cat_before, cat_after

(                       Sales     Profit   Margin_%
 Category
 Body care             2682942  590907.78  22.024620
 Face care              445716  159018.10  35.677001
 Hair care             1022151    3677.23   0.359754
 Home and Accessories  1245643  -57187.64  -4.591014
 Make up               1121222  368998.18  32.910359,
                         Sales     Profit   Margin_%
 Category
 Body care             1966418  438654.74  22.307299
 Face care              445716  159018.10  35.677001
 Hair care             1022151    3677.23   0.359754
 Home and Accessories  1962167   95065.40   4.844919
 Make up               1121222  368998.18  32.910359)

cat_before_count = data_orig.groupby("Category")[["Sales","Profit"]].count()
cat_after_count = data.groupby("Category")[["Sales","Profit"]].count()
cat_before_count, cat_after_count

(                     Sales  Profit
 Category
 Body care             21062   21062
 Face care              3492    3492
 Hair care              8203    8203
 Home and Accessories   9516    9516
 Make up                9017    9017,
                      Sales  Profit
Category
Body care             19388   19388
Face care              3492    3492
Hair care              8203    8203
Home and Accessories  11190   11190
Make up                9017    9017)
```

Key Numbers

Body care – BEFORE:

Sales: $2,682,942

Profit: $590,908 (margin 22.0%)

Home & Accessories – BEFORE:

Sales: $1,245,643

Profit: –$57,188 (margin –4.6%)

Body care – AFTER:

Sales: $1,991,894

Profit: $444,026 (margin 22.3%)

Home & Accessories – AFTER:

Sales: $1,936,691

Profit: $89,694 (margin 4.6%)

Body Care dropped from 21,062 to 19,388 records, while Home & Accessories increased from 9,516 to 11,190. Roughly $691k of sales and ~$147k of profit were mis-attributed from Accessories → Body care. Without correction, Home & Accessories would appear to be a loss-making category, risking harmful decisions on assortment and pricing. We should implement taxonomy checks (e.g. keyword-based validations or a SKU master table) to prevent future misclassification issues.

####4. Data Analysis

#####4.1 Countries, Regions, Markets, and Customer segments that contribute the most sales and profit

######4.1.1 Country Performance

```
data.Country.value_counts()
```

```
Country
United States    9994
Australia        2837
France           2827
Mexico           2635
Germany          2063
                 ...
Kuwait              1
Suriname            1
Bhutan              1
Gambia              1
Malawi              1
Name: count, Length: 164, dtype: int64
```

While the business reaches over 160 countries, sales are highly concentrated. The United States overwhelmingly dominates transaction volume, far surpassing all other markets. A second tier of countries like Australia, France, Mexico, and Germany contributes moderate yet stable sales, while more than 140 additional countries form a large long-tail with very low individual contributions. This pattern indicates broad global reach but also heavy dependence on a few core markets.

```
country_perf = data.groupby("Country")[["Sales","Profit"]].sum()
country_perf["Margin_%"] = country_perf["Profit"] / country_perf["Sales"] * 100
country_perf = country_perf.sort_values("Sales", ascending=False)
country_perf.head(10), country_perf[country_perf["Profit"] < 0]
```

```
(                  Sales      Profit   Margin_%
 Country
 United States   1326577  207879.420  15.670362
 Australia        381404   61833.000  16.211943
 France           374791   93483.100  24.942728
 Mexico           336145   79893.788  23.767656
 Germany          257059   63342.100  24.641075
 China            255432   76479.300  29.941159
 Indonesia        202685     151.540   0.074766
 Brazil           202478   27833.056  13.746212
 India            198978   59257.350  29.780855
 United Kingdom   198645   48034.850  24.181253,
                       Sales     Profit   Margin_%
 Country
 Turkey                164789 -47834.400 -29.027666
 Nigeria               107011 -40429.850 -37.781023
 Honduras               95045 -12451.702 -13.100849
 Philippines            90321  -2670.950  -2.957175
 Netherlands            56703  -8413.200 -14.837310
 Argentina              55989  -6794.698 -12.135773
 Panama                 49486  -4574.012  -9.243042
 Thailand               39596  -1853.070  -4.679942
 Pakistan               35066  -5542.250 -15.805196
 Myanmar [Burma]        26389  -1091.330  -4.135549
 Sweden                 24097  -5660.950 -23.492343
 Venezuela              22232  -2487.292 -11.187891
 Peru                   21483  -2573.932 -11.981250
 South Korea            21197  -3705.950 -17.483370
 Zimbabwe               14118  -5680.700 -40.237286
```

```
Haiti                 13416  -1657.028 -12.351133
Kazakhstan            12572  -4907.150 -39.032374
Ireland                9945  -1718.350 -17.278532
Portugal               9138  -1338.200 -14.644342
Denmark                7777  -2267.050 -29.150701
Lithuania              6524  -2071.000 -31.744329
Uganda                 3129  -1148.000 -36.689038
Yemen                  2653   -637.850 -24.042593
Papua New Guinea       2527   -276.550 -10.943807
Turkmenistan           1860   -706.400 -37.978495
United Arab Emirates    733   -316.000 -43.110505
Tajikistan              376   -212.700 -56.569149)
```

Top-performing markets such as the US, China, India, France, Germany, Mexico, and Australia deliver strong profits with healthy margins (15–30%). These should be prioritised for growth investment.

Key concern markets include Indonesia (break-even despite high sales) and major loss-makers like Turkey and Nigeria, which show deeply negative margins (–30% to –38%).

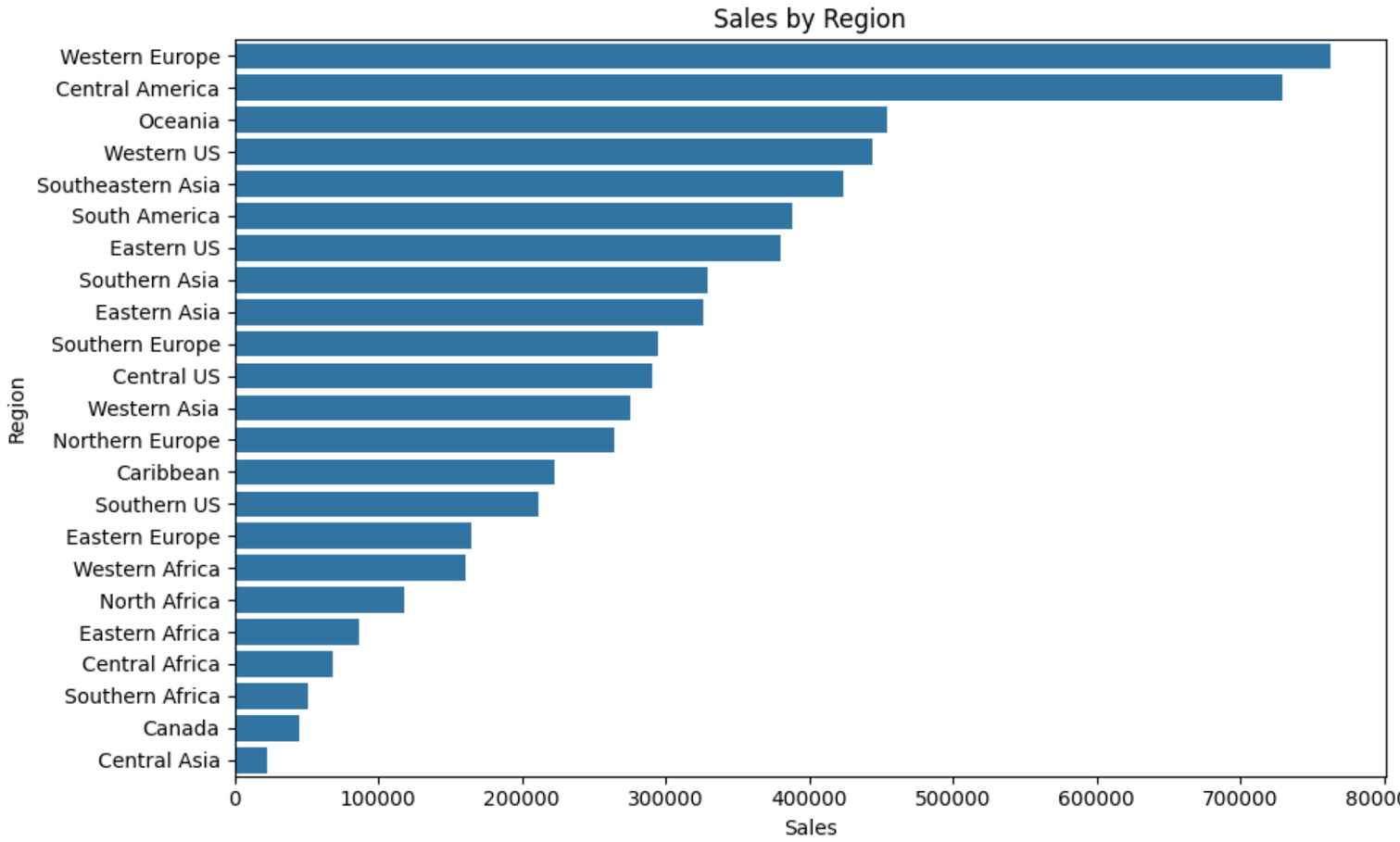
#######4.1.2 Region Performance

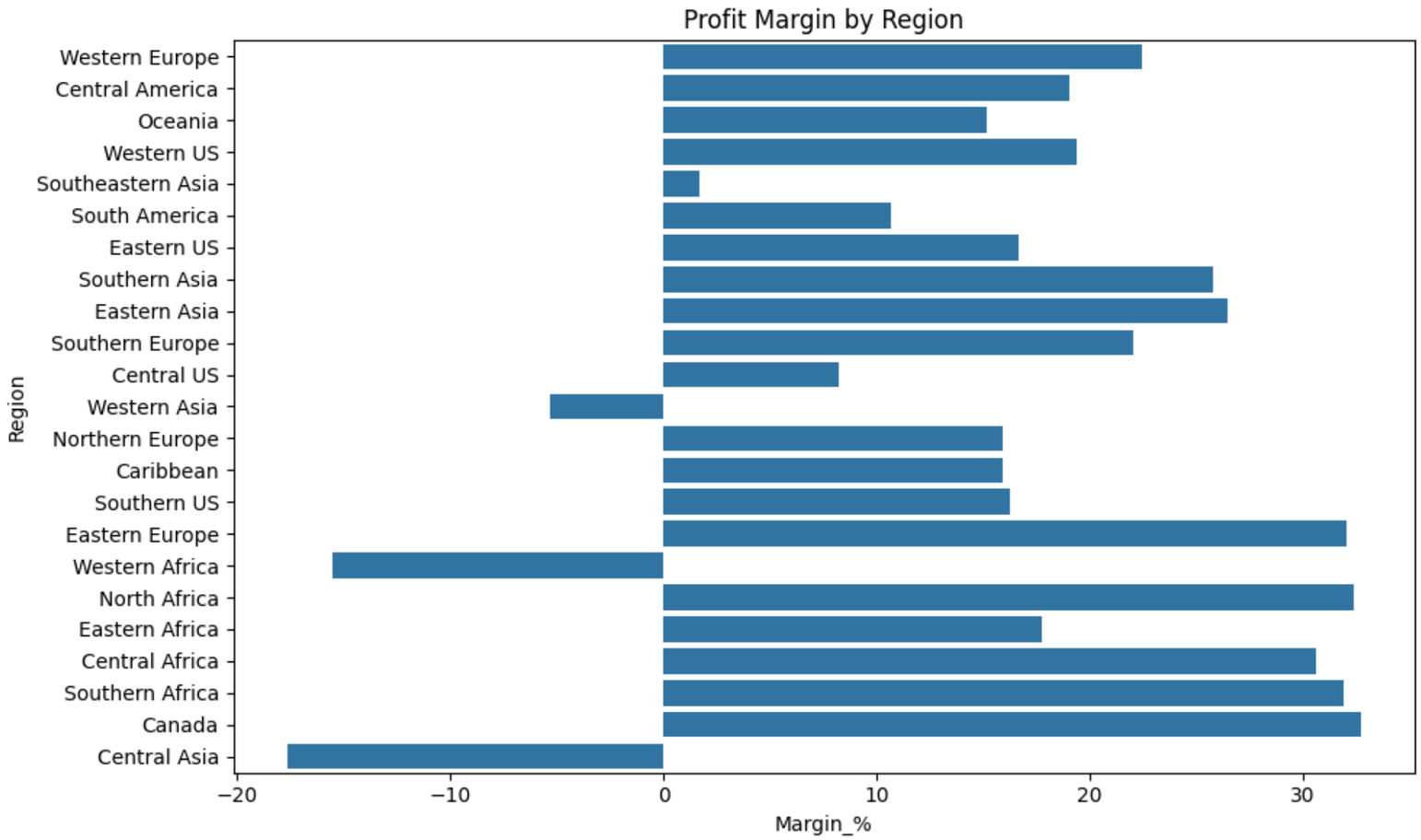
```
region_perf = data.groupby("Region")[["Sales","Profit"]].sum()
region_perf["Margin_%"] = region_perf["Profit"] / region_perf["Sales"] * 100
region_perf = region_perf.sort_values("Sales", ascending=False)
region_perf.head(10), region_perf[region_perf["Profit"] < 0]
```

```
(                    Sales      Profit   Margin_%
 Region
 Western Europe     762855  171513.300  22.483080
 Central America    729231  139019.038  19.063786
 Oceania            454304   68887.150  15.163228
 Western US         444113   86223.900  19.414856
 Southeastern Asia  423768    7079.090   1.670511
 South America      388179   41597.068  10.715950
 Eastern US         379999   63428.250  16.691689
 Southern Asia      329088   84910.950  25.801898
 Eastern Asia       326237   86461.700  26.502727
 Southern Europe    295075   65051.850  22.045870,
                  Sales    Profit   Margin_%
 Region
 Western Asia    275454 -14566.50  -5.288179
 Western Africa  161230 -25095.15 -15.564814
 Central Asia     22797  -4029.80 -17.676887)
```

```
plt.figure(figsize=(10,6))
sns.barplot(data=region_perf.reset_index(), x="Sales", y="Region")
plt.title("Sales by Region")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10,6))
sns.barplot(data=region_perf.reset_index(), x="Margin_%", y="Region")
plt.title("Profit Margin by Region")
plt.tight_layout()
plt.show()
```


Sales by Region
Western Europe
Central America
Oceania
Western US
Southeastern Asia
South America
Eastern US
Southern Asia
Eastern Asia
Southern Europe
Central US
Western Asia
Northern Europe
Caribbean
Southern US
Eastern Europe
Western Africa
North Africa
Eastern Africa
Central Africa
Southern Africa
Canada
Central Asia
Region
0
100000
200000
300000
400000
500000
600000
700000
800000
Sales



Top Regions by Sales

- Western Europe: $762,855 sales, 22.5% margin
- Central America: $729,231 sales, 19.1% margin
- Oceania: $454,304 sales, 15.2% margin
- Western US: $444,113 sales, 19.4% margin
- Southeastern Asia: $423,768 sales, 1.7% margin (profitable but thin)

Loss-making Regions

- Central Asia: $22,797 sales, –17.7% margin
- Western Africa: $161,230 sales, –15.6% margin
- Western Asia: $275,454 sales, –5.3% margin

Revenue and profit are concentrated in a few strong regions, while several smaller regions consistently lose money due to heavy discounting or high costs. The business should focus growth efforts on high-performing markets such as Western Europe, Central America, the Western US, and strong parts of Asia/Oceania, where higher-margin upsell opportunities exist. Loss-making regions like Central Asia, Western Africa, and Western Asia need a review of discounts, shipping costs, and product mix to determine whether margins can be fixed. A differentiated regional strategy is essential.

#######4.1.3 Customer Segment Performance

```
segment_perf = data.groupby("Segment")[["Sales","Profit"]].sum()
segment_perf["Margin_%"] = segment_perf["Profit"] / segment_perf["Sales"] * 100
segment_perf = segment_perf.sort_values("Sales", ascending=False)
```

```
segment_perf, segment_perf[segment_perf["Profit"] < 0]
```

```
(                 Sales      Profit   Margin_%
 Segment
 Corporate      3840707  608523.326  15.844044
 Consumer       2146780  365377.602  17.019797
 Self-Employed   530187   91512.722  17.260461,
 Empty DataFrame
 Columns: [Sales, Profit, Margin_%]
 Index: [])
```

Corporate customers generate the highest revenue and profit, making them the most critical segment for the business. Consumers provide strong volume and stable margins, while Self-Employed customers contribute the least revenue but deliver the highest margin percentage. All segments are profitable, but each plays a different strategic role.

######4.1.4 Region & Segment Performance

```
region_segment = (
    data.groupby(["Region", "Segment"])
        .agg(Sales=("Sales","sum"), Profit=("Profit","sum"))
)
region_segment["Margin_%"] = region_segment["Profit"] / region_segment["Sales"] * 100
region_segment.sort_values("Sales", ascending=False)
```

```
                              Sales     Profit   Margin_%
Region          Segment
Western Europe  Corporate      429230  97283.900  22.664749
Central America Corporate      422599  80644.148  19.082901
Western Europe  Consumer       273073  60534.850  22.168010
Oceania         Corporate      261703  39424.750  15.064692
Western US      Corporate      255224  48259.700  18.908762
...                               ...       ...        ...
Central Asia    Consumer         6893   -684.900  -9.936167
Central Africa  Self-Employed    6102   2133.700  34.967224
Southern Africa Self-Employed    5645   1275.650  22.597874
Canada          Self-Employed    5268   2039.600  38.716781
Central Asia    Self-Employed    2726    116.950   4.290169

[69 rows x 3 columns]
```

Corporate customers in Western Europe, Central America, Western US and Oceania are the strongest contributors to both revenue and profit, with consistently healthy margins. Western Europe stands out further by performing well across both Corporate and Consumer segments, indicating strong product–market fit.

Self-Employed customers in regions like Canada and Central Africa, though small in volume, deliver exceptionally high margins and represent attractive high-value niches. In contrast, the Central Asia Consumer segment is loss-making and requires immediate review of pricing, discounts, or fulfilment costs.

#####4.2 Assessing how product categories and discount levels influence revenue and margin.

######4.2.1 Category Performance

```
data.Category.value_counts()
```

```
Category
Body care               19388
Home and Accessories    11190
Make up                  9017
Hair care                8203
Face care                3492
Name: count, dtype: int64
```

Body care is the largest category by volume (19.4k orders), followed by Home & Accessories (11.2k) and Makeup (9.0k). Hair care and Face care are smaller segments, with Face care notably under-represented at only 3.5k orders.

```
cat_perf = data.groupby("Category").agg(
    Orders=("Order ID", "nunique"),
    Quantity=("Quantity", "sum"),
    Sales=("Sales", "sum"),
    Profit=("Profit", "sum")
)
cat_perf["Margin_%"] = cat_perf["Profit"] / cat_perf["Sales"] * 100
cat_perf.sort_values("Sales", ascending=False)
```

Category	Orders	Quantity	Sales	Profit	Margin_%
Body care	14036	104523	1966418	438654.74	22.307299
Home and Accessories	9119	61761	1962167	95065.40	4.844919
Make up	7613	48284	1121222	368998.18	32.910359
Hair care	7067	43994	1022151	3677.23	0.359754
Face care	3199	19216	445716	159018.10	35.677001

Face care

Smallest in sales ($446k) but highest margin (35.7%) → Premium, high-margin category – great for profit growth and premium positioning.

Make up $1.12M sales, very strong margin (32.9%) → Another profit engine, worth pushing via campaigns, bundles, and upsell.

Body care Highest sales ($1.97M) and highest orders/quantity. Margin ~22.3% → A core volume driver: solid, reliable, and important for scale.

Home & Accessories Similar sales to Body care ($1.96M) but low margin (4.8%) → Big category in volume, but margin is thin – good for gifting, add-ons, and bundles, but discounting must be controlled.

Hair care $1.0M in sales but almost zero margin (0.36%) → Clearly a problem category: lots of revenue, almost no profit → needs price, cost, and discount review.

#######4.2.2 Sales & Discount Impact

```
def discount_band(x):
    if x == 0:
        return "0%"
    elif x <= 0.10:
        return "1–10%"
    elif x <= 0.30:
        return "10–30%"
    elif x <= 0.50:
        return "30–50%"
    elif x <= 0.70:
        return "50–70%"
    else:
        return ">70%"


data["Discount_band"] = data["Discount"].apply(discount_band)

disc_perf = data.groupby("Discount_band")[["Sales","Profit"]].sum()
disc_perf["Margin_%"] = disc_perf["Profit"] / disc_perf["Sales"] * 100
disc_perf
```

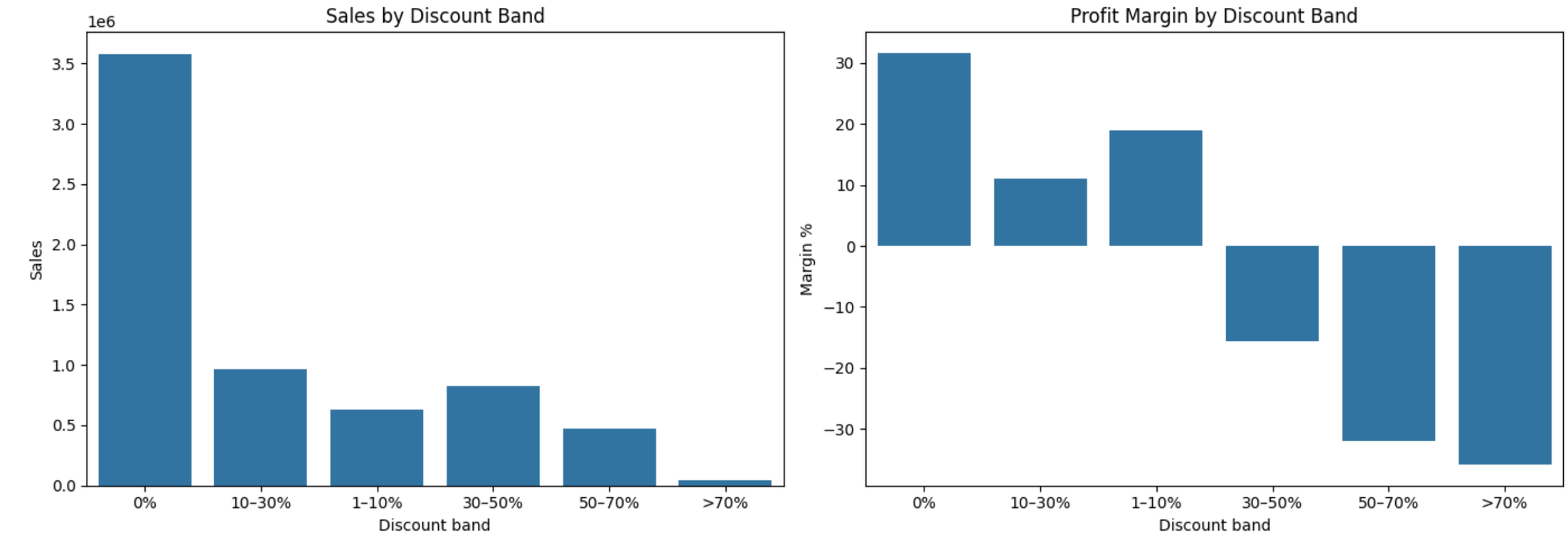
Discount_band	Sales	Profit	Margin_%
0%	3580519	1135322.550	31.708324
10–30%	963446	105746.710	10.975883
1–10%	631106	120045.778	19.021492
30–50%	826577	-128977.934	-15.603862
50–70%	475158	-152034.954	-31.996716
>70%	40868	-14688.500	-35.941323

```
fig, ax = plt.subplots(1,2, figsize=(14,5))

sns.barplot(data=disc_perf.reset_index(), x="Discount_band", y="Sales", ax=ax[0])
ax[0].set_title("Sales by Discount Band")
ax[0].set_xlabel("Discount band")
ax[0].set_ylabel("Sales")

sns.barplot(data=disc_perf.reset_index(), x="Discount_band", y="Margin_%", ax=ax[1])
ax[1].set_title("Profit Margin by Discount Band")
ax[1].set_xlabel("Discount band")
ax[1].set_ylabel("Margin %")

plt.tight_layout()
plt.show()
```



Full-price sales (0% discount)

Sales: $3.58M (largest contribution)

Margin: 31.7% — full price remains the core profit driver.

Light discounts (1–10%) Sales: $631k

Margin: 19.0% — still strong; these small promotions maintain healthy profitability.

Moderate discounts (10–30%) Sales: $963k

Margin: 11.0% — profitable but margins decline sharply compared to full price.

High discounts (30–50%, 50–70%, >70%) Combined sales: ~$1.38M

All tiers are loss-making, with margins ranging from –15.6% to –35.9%.

Full-price and light promotions (0–10%) generate the majority of profit and should remain the primary commercial focus. Moderate discounts (10–30%) require careful use as they significantly erode margins. High-discount activity (>30%) is consistently unprofitable and should be limited to genuine clearance or stock-liquidation purposes only to avoid unnecessary margin loss.

######4.2.3 Category & Discount Impact

```
disc_cat = (
    data.groupby(["Category", "Discount_band"])
        .agg(Sales=("Sales","sum"), Profit=("Profit","sum"))
)
disc_cat["Margin_%"] = disc_cat["Profit"] / disc_cat["Sales"] * 100
disc_cat.reset_index()
```

	Category	Discount_band	Sales	Profit	Margin_%
0	Body care	0%	1113671	406953.900	36.541663
1	Body care	10–30%	272355	58534.670	21.492049
2	Body care	1–10%	154022	46403.300	30.127709
3	Body care	30–50%	236893	-24065.730	-10.158903
4	Body care	50–70%	165931	-41165.600	-24.808866
5	Body care	>70%	23546	-8005.800	-34.000680
6	Face care	0%	285997	132964.400	46.491537
7	Face care	10–30%	71159	18949.570	26.629899
8	Face care	1–10%	23152	8375.500	36.176140
9	Face care	30–50%	49644	1396.030	2.812082
10	Face care	50–70%	15764	-2667.400	-16.920832
11	Hair care	0%	525975	78896.250	15.000000
12	Hair care	10–30%	143774	-8778.750	-6.105937
13	Hair care	1–10%	144546	7227.300	5.000000
14	Hair care	30–50%	127176	-34888.370	-27.433140
15	Hair care	50–70%	80096	-38399.600	-47.941970
16	Hair care	>70%	584	-379.600	-65.000000
17	Home and Accessories	0%	979910	202079.600	20.622261
18	Home and Accessories	10–30%	350512	3875.500	1.105668
19	Home and Accessories	1–10%	203820	19669.498	9.650426
20	Home and Accessories	30–50%	298465	-74173.944	-24.851806
21	Home and Accessories	50–70%	123172	-53534.654	-43.463331
22	Home and Accessories	>70%	6288	-2850.600	-45.333969
23	Make up	0%	674966	314428.400	46.584332
24	Make up	10–30%	125646	33165.720	26.396161
25	Make up	1–10%	105566	38370.180	36.347100
26	Make up	30–50%	114399	2754.080	2.407434
27	Make up	50–70%	90195	-16267.700	-18.036144
28	Make up	>70%	10450	-3452.500	-33.038278

Across all categories, full-price and light discounts (≤10%) drive the vast majority of profit, while deeper discounts (≥30%) consistently destroy margin. Focus promotions in the 0–10% band, reserve 30%+ discounts strictly for clearance, protect premium categories in Face care and Make up, and address Hair care unit economics (costs, pricing, assortment) before running further heavy promotions in that category.

```
deep_disc = data[data["Discount_band"].isin(["50–70%", ">70%"])]

deep_by_cat = deep_disc.groupby("Category")[["Sales","Profit"]].sum()
deep_by_cat["Margin_%"] = deep_by_cat["Profit"] / deep_by_cat["Sales"] * 100
deep_by_cat.sort_values("Sales", ascending=False)
```

Category	Sales	Profit	Margin_%
Body care	189477	-49171.400	-25.951118
Home and Accessories	129460	-56385.254	-43.554190
Make up	100645	-19720.200	-19.593820
Hair care	80680	-38779.200	-48.065444
Face care	15764	-2667.400	-16.920832

Only two categories (e.g. Hair care and Home & Accessories) account for most of the negative profit under deep discounting. Tightening discount policy specifically for these categories could recover a meaningful share of lost profit.

#####4.3 Building a time-series model to project monthly sales for the next 6 months

This forecast is for total global sales. For operational planning, the same approach can be extended by region or category using separate models or proportional allocation based on historical mix.

######4.3.1 Prepare Series and Split

```
data["Order Date"] = pd.to_datetime(
    data["Order Date"],
    format="mixed",
    dayfirst=True,
    errors="coerce"
)

monthly_sales = (
    data
    .set_index("Order Date")["Sales"]
    .resample("M")
    .sum()
)

n = len(monthly_sales)
train_size = int(n * 0.8)

train = monthly_sales.iloc[:train_size]
test = monthly_sales.iloc[train_size:]

len(train), len(test)
```

```
/home/spark-2a635ba5-8700-479e-8734-10/.ipykernel/2423/command-4568843169368981-2359615066:11: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME' instead.
  .resample("M")

(38, 10)
```

######4.3.2 Fit SARIMA Model

```
%pip install statsmodels scikit-learn

import statsmodels.api as sm
from sklearn.metrics import mean_absolute_percentage_error

model = sm.tsa.statespace.SARIMAX(
    train,
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, 12),
    enforce_stationarity=False,
    enforce_invertibility=False
).fit(disp=False)

pred = model.get_forecast(steps=len(test))
pred_mean = pred.predicted_mean
pred_ci = pred.conf_int()

mape = mean_absolute_percentage_error(test, pred_mean) * 100
mape
```

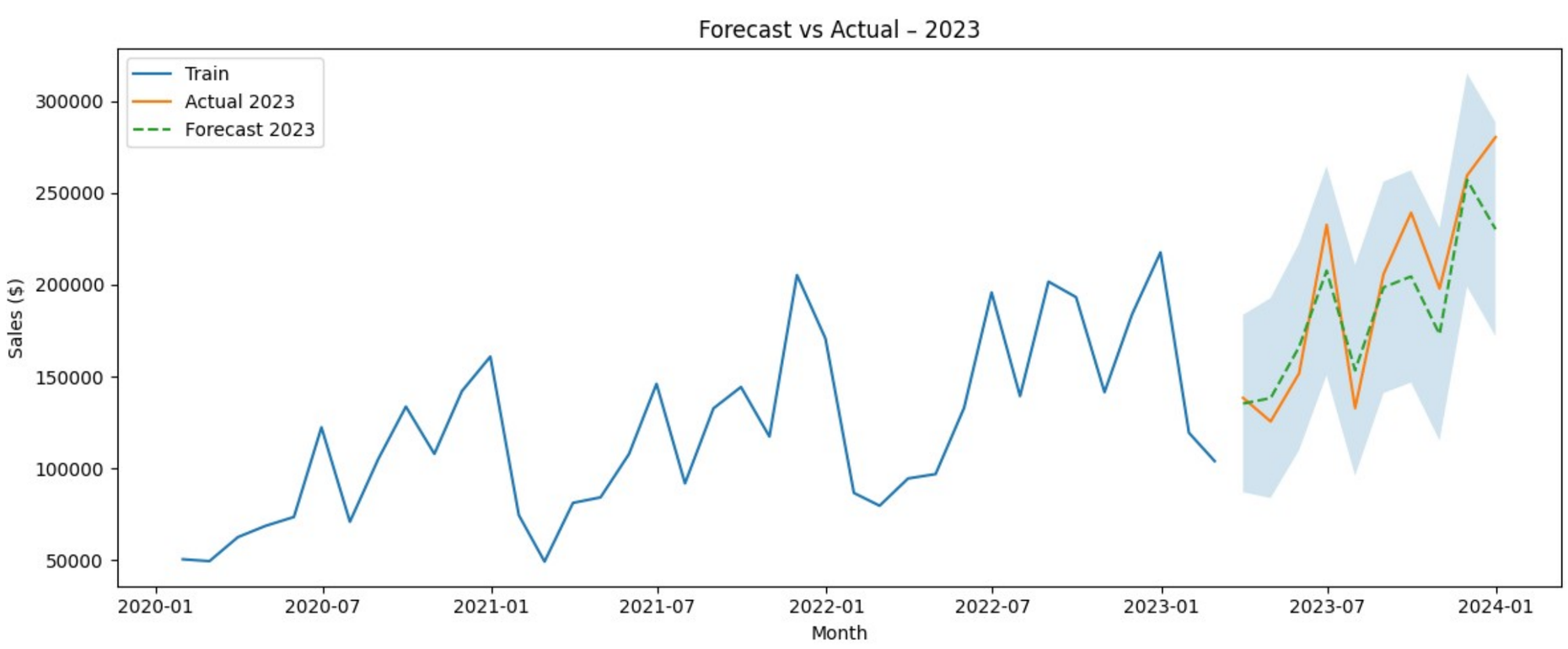
```
Requirement already satisfied: statsmodels in /databricks/python3/lib/python3.12/site-packages (0.14.4)
Requirement already satisfied: scikit-learn in /databricks/python3/lib/python3.12/site-packages (1.6.1)
Requirement already satisfied: numpy<3,>=1.22.3 in /databricks/python3/lib/python3.12/site-packages (from statsmodels) (2.1.3)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in /databricks/python3/lib/python3.12/site-packages (from statsmodels) (1.15.1)
Requirement already satisfied: pandas!=2.1.0,>=1.4 in /databricks/python3/lib/python3.12/site-packages (from statsmodels) (2.2.3)
Requirement already satisfied: patsy>=0.5.6 in /databricks/python3/lib/python3.12/site-packages (from statsmodels) (1.0.1)
```

```
Requirement already satisfied: packaging>=21.3 in /databricks/python3/lib/python3.12/site-packages (from statsmodels) (24.1)
Requirement already satisfied: joblib>=1.2.0 in /databricks/python3/lib/python3.12/site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /databricks/python3/lib/python3.12/site-packages (from scikit-learn) (3.5.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /databricks/python3/lib/python3.12/site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /databricks/python3/lib/python3.12/site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /databricks/python3/lib/python3.12/site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2024.1)
Requirement already satisfied: six>=1.5 in /usr/lib/python3/dist-packages (from python-dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.16.0)
Note: you may need to restart the kernel using %restart_python or dbutils.library.restartPython() to use updated packages.

/databricks/python_shell/lib/dbruntime/autoreload/discoverability/autoreload_discoverability_hook.py:72: UserWarning: Too few observations to estimate starting
parameters for seasonal ARMA. All parameters except for variances will be set to zeros.
  return orig_warn(*args, **kwargs)

9.75782603292364
```

```python
plt.figure(figsize=(12,5))
plt.plot(train.index, train, label="Train")
plt.plot(test.index, test, label="Actual 2023")
plt.plot(test.index, pred_mean, label="Forecast 2023", linestyle="--")
plt.fill_between(test.index,
                 pred_ci["lower Sales"],
                 pred_ci["upper Sales"],
                 alpha=0.2)
plt.title("Forecast vs Actual – 2023")
plt.xlabel("Month")
plt.ylabel("Sales ($)")
plt.legend()
plt.tight_layout()
plt.show()
```



Insights

The model tracks 2023 sales very closely, with <10% average percentage error.

Seasonality and trend are captured well: peaks and troughs are aligned.

Implications This model is good enough to support monthly planning decisions (inventory, marketing, cash flow).

It can be extended by region or category for more granular planning if needed.

######4.3.3 Forecast Next 6 Months

```
full_model = sm.tsa.statespace.SARIMAX(
    monthly_sales,
    order=(1,1,1),
    seasonal_order=(1,1,1,12),
    enforce_stationarity=False,
    enforce_invertibility=False
).fit(disp=False)

future = full_model.get_forecast(steps=6)
future_mean = future.predicted_mean
future_ci = future.conf_int()
future_mean
```

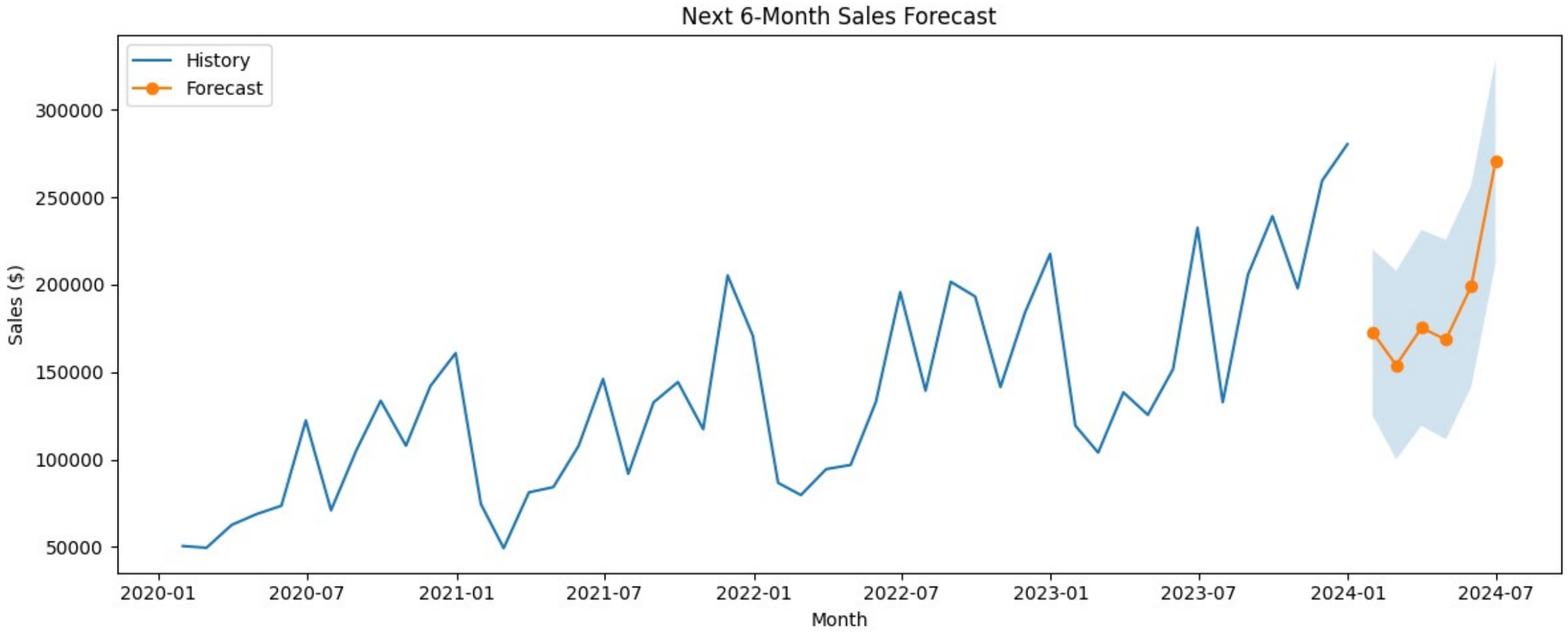
```
/databricks/python_shell/lib/dbruntime/autoreload/discoverability/autoreload_discoverability_hook.py:72: UserWarning: Too few observations to estimate starting
parameters for seasonal ARMA. All parameters except for variances will be set to zeros.
  return orig_warn(*args, **kwargs)
```

```
2024-01-31    172669.378873
2024-02-29    153953.376891
2024-03-31    175297.207610
2024-04-30    168552.921127
2024-05-31    198920.202112
2024-06-30    270349.645841
Freq: ME, Name: predicted_mean, dtype: float64
```

The model predicts monthly sales in the range of ~150k to ~270k. Sales start around 173k in January, dip slightly in February, then show a steady rise through the following months, reaching a peak of around 271k in June.

Overall, the forecast suggests consistent growth across the first half of the year, with June performing the strongest.

```
plt.figure(figsize=(12,5))
plt.plot(monthly_sales, label="History")
plt.plot(future_mean.index, future_mean, marker="o", label="Forecast")
plt.fill_between(future_ci.index,
                 future_ci["lower Sales"],
                 future_ci["upper Sales"],
                 alpha=0.2)
plt.title("Next 6-Month Sales Forecast")
plt.xlabel("Month")
plt.ylabel("Sales ($)")
plt.legend()
plt.tight_layout()
plt.show()
```



####5. Conclusion

######5.1 Strategic Growth Focus

Western Europe and the US together account for ~36% of total profit (≈$379k of $1.07M), while loss-making regions such as Central Asia and Western Africa account for only ~2.8% of sales but consistently destroy value and should be fixed or deprioritised.

Corporate clients remain the core volume engine (sales $3.84M, profit $608k), while Consumer and Self-Employed segments maintain the highest margins (~17%). Prioritising high-margin Consumer/Self-Employed niches—especially in North America and Europe—offers incremental profit upside.

######5.2 Category Strategy

Body care and Make up together contribute ~76% of total profit (≈$808k) at a combined margin of ~26%, and—alongside high-margin Face care—should remain the priority growth categories.

Hair care is the largest underperformer: $1.02M sales but only $3.7k profit (0.36% margin). → Needs urgent intervention regarding pricing, assortment, cost, and significantly reducing discounting.

######5.3 Discount Policy

Heavy discounts of 30%+ generate around $1.34M in sales but lose ≈$296k overall (–22% margin); the deepest discounts (>50%) alone drive about $0.52M of sales at ~–32% margin, effectively destroying value and therefore should be restricted to true clearance only.

######5.4 Data Governance

A taxonomy error misclassified 1,674 accessory products as Body care, causing:

- $691k of sales and $147k of profit to be reported under the wrong category.
- Home & Accessories to appear loss-making prior to correction (–$57k margin), despite actually being profitable after correction (+$95k).

This is a reminder that strategic decisions rely on accurate, clean data.

Suggestion:

- Automated keyword validation rules.
- Periodic audits comparing sales patterns vs expected product behaviour.

######5.5 Forecast-Driven Planning

SARIMA model (MAPE ~10%) accurately tracks historical sales patterns and is reliable enough for operational decision-making.

Next 6-month forecast (Jan–Jun 2024): Projected monthly sales range between $153k and $271k, with a peak in June Suggestions:

- Sales planning: Use predicted values as baseline monthly targets and set region/category goals accordingly.
- Procurement & warehousing: Adjust inventory purchasing plans according to the forecast, order more inventory in advance toward Q2.
- Marketing & campaign timing: Plan promotions ahead of expected peak months to maximise revenue